

各种筛选方法的比较

提供几种实现方法，按 `city` 分组并筛选出 `Revenue` 均值大于 200 的组对应的所有行。

推荐方法

方法1：使用 `transform()`（最常用、最高效）

代码块

```
1  import pandas as pd
2
3  # 示例数据
4  df = pd.DataFrame({
5      'city': ['北京', '上海', '北京', '广州', '上海', '北京', '广州', '上海'],
6      'Revenue': [300, 150, 250, 180, 220, 280, 190, 240],
7      'Product': ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H']
8  })
9
10 print("原始数据:")
11 print(df)
12 print()
13
14 # ===== 核心代码 =====
15 # 计算每个城市的 Revenue 均值，并广播到每一行
16 city_mean = df.groupby('city')['Revenue'].transform('mean')
17 print("每个城市对应的 Revenue 均值:")
18 print(city_mean)
19 print()
20
21 # 筛选出均值 > 200 的城市的所有行
22 result = df[city_mean > 200]
23 print("筛选结果 (Revenue均值>200的城市的所有行) :")
24 print(result)
```

输出：

代码块

1 原始数据:

```
2     city  Revenue Product
3  0   北京      300      A
4  1   上海      150      B
5  2   北京      250      C
6  3   广州      180      D
7  4   上海      220      E
8  5   北京      280      F
9  6   广州      190      G
10 7    上海      240      H
```

11

12 每个城市对应的 Revenue 均值:

```
13 0    276.67
14 1    203.33
15 2    276.67
16 3    185.00
17 4    203.33
18 5    276.67
19 6    185.00
20 7    203.33
```

21

22 筛选结果:

```
23     city  Revenue Product
24  0   北京      300      A
25  1   上海      150      B
26  2   北京      250      C
27  4   上海      220      E
28  5   北京      280      F
29  7   上海      240      H
```

其他实现方法

方法2: 使用 `filter()`

代码块 *filter* 直接对分组进行过滤

```
2 result = df.groupby('city').filter(lambda x: x['Revenue'].mean() > 200)
3 print("使用 filter 方法:")
4 print(result)
```

优点：代码简洁

缺点：大数据量时性能较差（因为要对每个组应用 lambda）

方法3：先计算均值，再合并

代码块

```
1 # 步骤1: 计算每个城市的均值
2 city_avg = df.groupby('city')['Revenue'].mean().reset_index()
3 city_avg.columns = ['city', 'avg_revenue']
4 print("各城市平均收入:")
5 print(city_avg)
6
7 # 步骤2: 筛选出均值 > 200 的城市
8 qualified_cities = city_avg[city_avg['avg_revenue'] > 200]['city']
9 print("\n符合条件的城市:")
10 print(qualified_cities.tolist())
11
12 # 步骤3: 从原数据中筛选这些城市的行
13 result = df[df['city'].isin(qualified_cities)]
14 print("\n筛选结果:")
15 print(result)
```

优点：逻辑清晰，易于理解

缺点：代码较长，需要多步操作

方法4：使用 `merge()`

代码块

```
1  # 计算均值并标记符合条件的城市
2  city_stats = df.groupby('city')['Revenue'].mean().reset_index()
3  city_stats['qualified'] = city_stats['Revenue'] > 200
4
5  # 合并回原数据
6  result = df.merge(city_stats[['city', 'qualified']], on='city')
7  result = result[result['qualified']].drop(columns=['qualified'])
8
9  print("使用 merge 方法:")
10 print(result)
```

完整实战示例

代码块

```
1  import pandas as pd
2  import numpy as np
3
4  # 创建更真实的销售数据
5  np.random.seed(42)
6  df = pd.DataFrame({
7      'city': np.random.choice(['北京', '上海', '广州', '深圳', '杭州'], 100),
8      'Revenue': np.random.randint(100, 500, 100),
9      'Sales': np.random.randint(10, 100, 100),
10     'Date': pd.date_range('2024-01-01', periods=100, freq='D')
11 })
12
13 print("="*70)
14 print("城市销售数据分析")
```

```

15 print("="*70)
16
17 # 查看各城市的统计信息
18 print("\n【步骤1】各城市 Revenue 统计")
19 print("-"*70)
20 city_stats = df.groupby('city')['Revenue'].agg(['mean', 'sum', 'count'])
21 print(city_stats.round(2))
22
23 # 使用方法1: transform 筛选
24 print("\n【步骤2】筛选 Revenue 均值 > 200 的城市的所有订单")
25 print("-"*70)
26
27 # 核心代码 (一行搞定)
28 result = df[df.groupby('city')['Revenue'].transform('mean') > 200]
29
30 print(f"原始数据行数: {len(df)}")
31 print(f"筛选后行数: {len(result)}")
32 print(f"筛选比例: {len(result)/len(df)*100:.1f}%")
33 print("\n筛选后的数据预览:")
34 print(result.head(10))
35
36 # 验证: 查看筛选后包含哪些城市
37 print("\n【步骤3】验证筛选结果")
38 print("-"*70)
39 print("筛选后包含的城市:")
40 print(result['city'].unique())
41
42 print("\n筛选后各城市的 Revenue 均值:")
43 print(result.groupby('city')['Revenue'].mean().round(2))
44
45 # 对比: 被排除的城市
46 excluded_cities = set(df['city'].unique()) - set(result['city'].unique())
47 print(f"\n被排除的城市 (均值<=200): {excluded_cities}")
48 if excluded_cities:
49     for city in excluded_cities:
50         avg = df[df['city']==city]['Revenue'].mean()
51         print(f" - {city}: 均值={avg:.2f}")

```

方法	代码复杂度	性能	可读性	适用场景
transform	★ 简单	★★★★ 快	★★★★ 好	通用推荐 ✓
filter	★ 最简单	★★★ 中等	★★★★ 好	小数据集
分步筛选	★★★★ 复杂	★★★★ 快	★★★★★ 最好	需要中间结果
merge	★★ 中等	★★ 中等	★★ 一般	需要额外信息

🎯 最佳实践建议

推荐使用 `transform()` 的原因：

- 1. 性能好：向量化操作，避免 Python 循环
- 2. 代码简洁：一行代码搞定
- 3. 保留索引：不改变原数据的索引结构
- 4. 灵活性高：可以轻松组合多个条件

常见变体：

代码块

```
1  # 变体1: 多个条件
2  result = df[
3      (df.groupby('city')['Revenue'].transform('mean') > 200) &
4      (df.groupby('city')['Sales'].transform('mean') > 50)
5  ]
6
7  # 变体2: 使用其他聚合函数
8  result = df[df.groupby('city')['Revenue'].transform('median') > 200]
```

```
9 result = df[df.groupby('city')['Revenue'].transform('sum') > 1000]
10
11 # 变体3: 自定义函数
12 def custom_filter(x):
13     return x.mean() > 200 and x.std() < 100
14
15 result = df[df.groupby('city')['Revenue'].transform(custom_filter) > 0]
```

⚠ 注意事项

代码块

```
1 # 注意1: transform 返回的是与原数据等长的 Series
2 print(type(df.groupby('city')['Revenue'].transform('mean')))
3 # <class 'pandas.core.series.Series'>
4
5 # 注意2: 确保列名正确
6 # df.groupby('city')['Revenue'] ← Revenue 必须存在
7
8 # 注意3: 处理缺失值
9 df_clean = df.dropna(subset=['city', 'Revenue']) # 先清理缺失值
10 result = df_clean[df_clean.groupby('city')['Revenue'].transform('mean') > 200]
```

总结: 强烈推荐使用 `transform()` 方法:

代码块

```
1 result = df[df.groupby('city')['Revenue'].transform('mean') > 200]
```

这是最 Pythonic、最高效、最常用的写法！